

Epic Events - Architecture technique et sécurité

CRM en ligne de commande - Python, Peewee, SQLite, Typer, Sentry et Pytest

Projet OpenClassrooms P12

Juillet 2026

Sommaire

1 Synthèse de validation	2
2 Architecture overview	3
2.1 Purpose	3
2.2 Main components	3
2.3 Layer responsibilities	3
2.3.1 CLI layer	3
2.3.2 Authentication and authorization	3
2.3.3 Service layer	4
2.3.4 Persistence layer	4
2.3.5 Monitoring	4
2.4 Request flow	4
2.5 Project structure	5
2.6 Architectural properties	5
3 Database schema	6
3.1 Engine and ORM	6
3.2 Entity relationship diagram	6
3.3 Tables	7
3.3.1 users	7
3.3.2 clients	7
3.3.3 contracts	7
3.3.4 events	7
3.4 Relationship logic	8
3.5 Integrity and injection protection	8
4 Authentication and permissions	9
4.1 Authentication	9
4.2 Authorization	9
4.3 Permission matrix	9
4.4 Least privilege rules	10
4.4.1 Management	10
4.4.2 Commercial	10
4.4.3 Support	10
4.5 Account deactivation	10
5 Security and monitoring	11
5.1 Security controls	11
5.1.1 Password protection	11
5.1.2 SQL injection prevention	11
5.1.3 Data validation	11
5.1.4 Least privilege	11
5.1.5 Secret management	11
5.2 Sentry architecture	11
5.3 Controlled demonstration	12

5.4	Privacy-related settings	12
5.5	Expected versus unexpected errors	12
6	Testing strategy	14
6.1	Objectives	14
6.2	Test organization	14
6.3	Unit tests	14
6.4	Integration tests	14
6.5	Functional tests	14
6.6	Database isolation	15
6.7	Commands	15
6.8	Validated results	15
7	Installation and demonstration	17
7.1	Clean installation	17
7.1.1	Prerequisites	17
7.1.2	Clone	17
7.1.3	Virtual environment	17
7.1.4	Dependencies	17
7.1.5	Database	17
7.1.6	Initial management account	17
7.1.7	Help	17
7.2	Clean-environment validation checklist	17
7.3	Suggested 10-minute demonstration	18
7.3.1	1. Architecture and schema	18
7.3.2	2. Authentication	18
7.3.3	3. Management workflow	18
7.3.4	4. Commercial workflow	18
7.3.5	5. Support workflow	18
7.3.6	6. Security	18
7.3.7	7. Tests and coverage	19
7.3.8	8. Sentry	19
7.4	Final repository verification	19

1 Synthèse de validation

Le projet Epic Events met en œuvre une architecture en couches, une base relationnelle SQLite, l'authentification et l'autorisation par rôle, le principe du moindre privilège, la journalisation Sentry et une suite de tests automatisés.

Résultats validés :

- 147 tests réussis ;
- 0 échec ;
- 76 % de couverture globale ;
- événement de démonstration reçu dans Sentry ;
- base de tests SQLite isolée en mémoire ;
- secrets et base locale exclus du dépôt Git.

2 Architecture overview

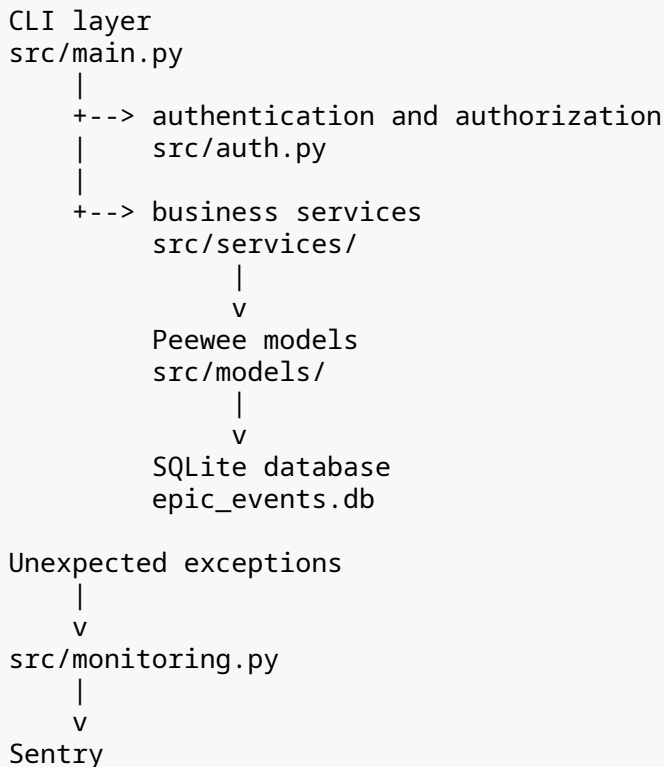
2.1 Purpose

Epic Events is a secure command-line CRM used by three departments:

- management;
- commercial;
- support.

The architecture separates terminal interaction, authentication, authorization, business rules, persistence and monitoring.

2.2 Main components



2.3 Layer responsibilities

2.3.1 CLI layer

`src/main.py`:

- defines Typer commands;
- collects terminal arguments;
- prompts securely for passwords;
- converts user input such as event dates;
- delegates operations to services;
- displays success and error messages;
- captures unexpected top-level exceptions.

Business rules must not be implemented in this layer.

2.3.2 Authentication and authorization

`src/auth.py`:

- authenticates users by email and password;
- verifies password hashes;

- checks active accounts;
- normalizes department names;
- exposes reusable role checks;
- raises explicit permission errors.

2.3.3 Service layer

src/services/:

- validates user data;
- enforces ownership, role and assignment rules;
- performs model queries and updates;
- hides restricted record existence where appropriate;
- remains independent from terminal presentation.

2.3.4 Persistence layer

src/models/ and src/database.py:

- define the relational model;
- configure the shared SQLite connection;
- use Peewee to generate parameterized SQL;
- preserve relationships through foreign-key constraints.

2.3.5 Monitoring

src/monitoring.py:

- initializes Sentry from environment variables;
- captures unexpected exceptions;
- flushes the transport queue;
- provides a controlled demonstration event;
- disables default personal-information and local-variable transmission.

2.4 Request flow

```
sequenceDiagram
    actor Collaborator
    participant CLI as Typer CLI
    participant Auth as Authentication
    participant Service as Service layer
    participant ORM as Peewee ORM
    participant DB as SQLite

    Collaborator->>CLI: command + public arguments
    CLI->>Collaborator: hidden password prompt
    CLI->>Auth: email + password
    Auth->>ORM: load User
    ORM->>DB: parameterized query
    DB-->>ORM: user row
    Auth-->>CLI: active authenticated User
    CLI->>Service: operation + current_user
    Service->>Service: role, ownership and input checks
    Service->>ORM: create/read/update
    ORM->>DB: parameterized SQL
    DB-->>ORM: result
    ORM-->>Service: model
    Service-->>CLI: result
    CLI-->>Collaborator: terminal response
```

2.5 Project structure

```
src/  
├── auth.py  
├── config.py  
├── database.py  
├── main.py  
├── monitoring.py  
├── models/  
│   ├── user.py  
│   ├── client.py  
│   ├── contract.py  
│   └── event.py  
└── services/  
    ├── user_service.py  
    ├── client_service.py  
    ├── contract_service.py  
    └── event_service.py
```

2.6 Architectural properties

- explicit separation of concerns;
- service-layer authorization;
- no direct SQL assembled from user input;
- no clear-text password storage;
- no secret hard-coded in source files;
- isolated test database;
- centralized unexpected-error monitoring;
- traceable technical decisions through ADR files.

3 Database schema

3.1 Engine and ORM

The application uses SQLite through Peewee.

SQLite was selected because the CRM is a local command-line application, the deployment target does not require a database server and the project remains easy to install in a clean environment.

Peewee provides:

- model-based schema definitions;
- parameterized queries;
- foreign-key relationships;
- readable service-layer operations;
- simple SQLite integration.

3.2 Entity relationship diagram

```
erDiagram
    USER ||--o{ CLIENT : "sales contact"
    USER ||--o{ CONTRACT : "sales contact"
    CLIENT ||--o{ CONTRACT : "owns"
    CONTRACT ||--o{ EVENT : "includes"
    USER o|--o{ EVENT : "support contact"

    USER {
        int id PK
        string full_name
        string email UK
        string password_hash
        string department
        boolean is_active
    }

    CLIENT {
        int id PK
        string full_name
        string email UK
        string phone
        string company_name
        datetime created_at
        datetime updated_at
        int sales_contact_id FK
    }

    CONTRACT {
        int id PK
        int client_id FK
        int sales_contact_id FK
        decimal total_amount
        decimal amount_due
        boolean is_signed
        datetime created_at
    }

    EVENT {
        int id PK
        int contract_id FK
        int support_contact_id FK
        string event_name
    }
```

```

    string location
    int attendees
    datetime event_start
    datetime event_end
    text notes
}

```

3.3 Tables

3.3.1 users

Field	Type	Constraint	Purpose
id	Integer	Primary key	Collaborator identifier
full_name	Varchar(255)	Required	Display name
email	Varchar(255)	Unique, required	Authentication identifier
password_hash	Varchar(255)	Required	Bcrypt password hash
department	Varchar(50)	Required	MANAGEMENT, COMMERCIAL or SUPPORT
is_active	Boolean	Default true	Logical account status

3.3.2 clients

Field	Type	Constraint	Purpose
id	Integer	Primary key	Client identifier
full_name	Varchar(255)	Required	Client name
email	Varchar(255)	Unique, required	Client email
phone	Varchar(30)	Required	Client phone
company_name	Varchar(255)	Required	Company
created_at	Datetime	Default current time	First contact
updated_at	Datetime	Updated by service	Last update
sales_contact_id	Integer	FK users, RESTRICT	Responsible commercial

3.3.3 contracts

Field	Type	Constraint	Purpose
id	Integer	Primary key	Contract identifier
client_id	Integer	FK clients, RESTRICT	Contract client
sales_contact_id	Integer	FK users, RESTRICT	Commercial contact
total_amount	Decimal(10,2)	Positive	Contract total
amount_due	Decimal(10,2)	0 to total	Remaining amount
is_signed	Boolean	Default false	Signature status
created_at	Datetime	Default current time	Creation timestamp

3.3.4 events

Field	Type	Constraint	Purpose
id	Integer	Primary key	Event identifier
contract_id	Integer	FK contracts, RESTRICT	Signed contract

Field	Type	Constraint	Purpose
support_contact_id	Integer	Nullable FK users, SET NULL	Assigned support
event_name	Varchar(255)	Required	Event label
location	Varchar(255)	Required	Venue
attendees	Integer	Greater than zero	Expected attendance
event_start	Datetime	Required	Start
event_end	Datetime	After start	End
notes	Text	Nullable	Operational notes

3.4 Relationship logic

- one commercial user may own many clients;
- one client may have many contracts;
- the contract stores the commercial contact associated with the client;
- one contract may have event records;
- an event may temporarily have no support collaborator;
- a support collaborator may be assigned to many events;
- account deactivation preserves related CRM records.

3.5 Integrity and injection protection

- user input is validated before persistence;
- unique email constraints prevent duplicates;
- amount and date consistency rules are enforced in services;
- foreign keys preserve entity relationships;
- Peewee compiles values as query parameters instead of concatenating raw SQL.

4 Authentication and permissions

4.1 Authentication

Every protected command requires the collaborator email and then requests the password through a hidden terminal prompt.

```
email argument
+
hidden password prompt
|
v
authenticate_user()
|
+--> unknown email: rejected
+--> invalid password: rejected
+--> inactive account: rejected
+--> valid active account: User returned
```

Passwords are:

- never stored in clear text;
- never passed as command-line arguments;
- hashed with Passlib and bcrypt;
- verified against the stored hash.

4.2 Authorization

Authorization is separate from authentication.

- authentication answers: "Who is the collaborator?"
- authorization answers: "May this collaborator perform this action?"

Role checks are centralized in `src/auth.py` and business-specific checks are applied in the service layer.

4.3 Permission matrix

Operation	Management	Commercial	Support
Read every client	Yes	Yes	Yes
Read every contract	Yes	Yes	Yes
Read every event	Yes	Yes	Yes
Create collaborator	Yes	No	No
Update collaborator	Yes	No	No
Deactivate collaborator	Yes	No	No
Create client	No	Yes	No
Update client	No	Own clients	No
Create contract	Yes	No	No
Update contract	All	Own-client contracts	No
List unsigned contracts	No	Yes	No
List unpaid contracts	No	Yes	No
Create event	No	Own client and signed contract	No
List events without support	Yes	No	No
Assign support to event	Yes	No	No
List assigned events	No	No	Own assignments
Update event	No	No	Own assignments
Trigger controlled Sentry test	Yes	No	No

4.4 Least privilege rules

4.4.1 Management

Management can administer collaborators, contracts and support assignment, but cannot create commercial clients or create operational events.

4.4.2 Commercial

A commercial collaborator:

- automatically owns every client they create;
- can update only their own clients;
- can update only contracts attached to their own clients;
- can create an event only for their own client;
- can create that event only when the contract is signed.

4.4.3 Support

A support collaborator:

- can filter events assigned to their account;
- can update only assigned events;
- cannot assign themselves to an event;
- cannot modify clients or contracts.

4.5 Account deactivation

Deleting a collaborator is implemented as logical deactivation.

This preserves:

- clients linked to former commercial users;
- contracts and their audit context;
- events and historical assignments;
- referential integrity.

A deactivated collaborator cannot authenticate or exercise permissions.

5 Security and monitoring

5.1 Security controls

5.1.1 Password protection

- Passlib and bcrypt are used for hashing and verification.
- Passwords are hidden during CLI entry.
- Passwords are not present in command history.
- Only hashes are stored in SQLite.

5.1.2 SQL injection prevention

The application uses Peewee ORM operations rather than SQL strings assembled from terminal input. Peewee parameterizes values before executing queries.

Service-layer validation remains necessary because parameterization protects SQL execution, not business consistency.

5.1.3 Data validation

The services validate:

- required text fields;
- unique emails;
- allowed departments;
- contract amounts;
- signed status;
- ownership rules;
- positive attendee counts;
- event date order;
- optional notes;
- active-user status.

5.1.4 Least privilege

Read and write access is granted by role, ownership and assignment.

Permission checks are applied before sensitive database lookups where appropriate.

5.1.5 Secret management

The repository must not contain:

- real passwords;
- a real Sentry DSN;
- .env;
- epic_events.db;
- coverage artifacts.

The .gitignore excludes local secrets, databases and generated reports.

5.2 Sentry architecture

```
SENTRY_DSN
SENTRY_ENVIRONMENT
  |
  v
src/config.py
  |
  v
initialize_sentry()
```

```

|
+--> no DSN: monitoring disabled safely
|
+--> DSN configured:
    sentry_sdk.init()
    send_default_pii=False
    include_local_variables=False

```

Unexpected exception flow:

```

run_app()
|
v
Type application
|
+--> expected validation or permission error:
    explicit CLI message
|
+--> unexpected exception:
    capture_application_exception()
    |
    v
    Sentry

```

5.3 Controlled demonstration

The management-only command:

```
python -m src.main test-sentry-command manager@epicevents.com
```

generates:

Epic Events controlled Sentry demonstration error.

The command validates:

- management authentication;
- Sentry configuration;
- exception capture;
- event delivery;
- returned Sentry event identifier.

5.4 Privacy-related settings

The SDK is initialized with:

```

send_default_pii=False
include_local_variables=False

```

These settings reduce accidental transmission of personal information and local variables.

5.5 Expected versus unexpected errors

Expected errors are not application crashes.

Examples:

- invalid credentials;
- missing required field;
- unauthorized operation;
- invalid amount;
- event end before event start.

These are displayed as controlled CLI responses.

Unexpected technical exceptions are captured centrally and reported to Sentry.

6 Testing strategy

6.1 Objectives

The test suite verifies:

- authentication;
- authorization;
- role and ownership restrictions;
- persistence through Peewee;
- input validation;
- CLI workflows;
- Sentry integration behavior;
- database initialization;
- regression safety.

6.2 Test organization

```
tests/
├── conftest.py
├── unit/
│   └── test_monitoring.py
├── integration/
│   ├── test_auth.py
│   ├── test_client_service.py
│   ├── test_contract_service.py
│   ├── test_database.py
│   ├── test_event_service.py
│   └── test_user_service.py
└── functional/
    └── test_cli.py
```

6.3 Unit tests

Unit tests isolate monitoring behavior with mocks.

Validated result:

```
4 passed
```

6.4 Integration tests

Integration tests combine:

- service functions;
- authentication;
- Peewee models;
- an in-memory SQLite database;
- shared fixtures.

Validated result:

```
134 passed
```

6.5 Functional tests

Functional tests execute the public Typer interface with `CliRunner`.

Validated paths include:

- CLI command registration;

- authentication success and failure;
- authorized consultation;
- denied operations;
- invalid event dates;
- Sentry authorization;
- missing Sentry configuration;
- successful Sentry command.

Validated result:

9 passed

6.6 Database isolation

Automated tests use:

```
SqliteDatabase(":memory:")
```

All model classes are temporarily bound to this database.

Consequences:

- `epic_events.db` is not opened by tests;
- every test starts with a clean schema;
- test data cannot leak into local demonstration data;
- the same real models and services remain exercised.

6.7 Commands

Complete suite:

```
python -m pytest -v
```

By category:

```
python -m pytest tests/unit -v
```

```
python -m pytest tests/integration -v
```

```
python -m pytest tests/functional -v
```

Coverage:

```
python -m pytest --cov=src --cov-report=term-missing --cov-report=html
```

6.8 Validated results

147 tests passed

0 failures

76% total coverage

Main coverage values:

Component	Coverage
Models	100%
Client service	100%
Contract service	100%
User service	100%
Authentication	97%
Event service	97%
Monitoring	95%
Main CLI	41%
Total	76%

The CLI file contains many display and error branches. Business rules and security-critical layers retain high coverage.

7 Installation and demonstration

7.1 Clean installation

7.1.1 Prerequisites

- Git;
- Python 3.12 recommended;
- internet access for dependency installation;
- optional Sentry project for the monitoring demonstration.

7.1.2 Clone

```
git clone https://github.com/AlNocquet/OC-Projet12-EpicEvents.git
cd OC-Projet12-EpicEvents
```

7.1.3 Virtual environment

```
python -m venv venv
```

Git Bash on Windows:

```
source venv/Scripts/activate
```

PowerShell:

```
.\venv\Scripts\Activate.ps1
```

macOS or Linux:

```
source venv/bin/activate
```

7.1.4 Dependencies

```
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

7.1.5 Database

```
python -m src.main initialize-database
```

7.1.6 Initial management account

```
python -m src.main create-initial-management-user-command "Morgan Manager" manager@ep
```

The password is entered and confirmed through hidden prompts.

7.1.7 Help

```
python -m src.main --help
```

7.2 Clean-environment validation checklist

- virtual environment created;
- requirements installed without error;
- database tables created;
- initial management account created;
- authentication succeeds;

- CLI help lists the CRM commands;
- complete test suite passes;
- coverage report is generated;
- no secret is committed.

7.3 Suggested 10-minute demonstration

7.3.1 1. Architecture and schema

Show:

- docs/architecture/architecture-overview.md;
- docs/architecture/database-schema.md;
- the four Peewee models;
- relationships and foreign-key behavior.

7.3.2 2. Authentication

Run:

```
python -m src.main authenticate-user-command manager@epicevents.com
```

Explain:

- hidden password;
- bcrypt hash verification;
- active-account check;
- difference between authentication and authorization.

7.3.3 3. Management workflow

Demonstrate:

- collaborator creation;
- contract creation;
- list of unassigned events;
- assignment of a support collaborator.

7.3.4 4. Commercial workflow

Demonstrate:

- client creation with automatic ownership;
- update of an owned client;
- contract filters;
- event creation for a signed contract.

7.3.5 5. Support workflow

Demonstrate:

- list of assigned events;
- update of an assigned event;
- denial when attempting to update another support collaborator's event.

7.3.6 6. Security

Explain:

- Peewee parameterization;
- service-layer permission checks;
- least privilege;
- password hashing;
- secret exclusion;

- soft deletion.

7.3.7 7. Tests and coverage

Run:

```
python -m pytest -v
python -m pytest --cov=src --cov-report=term-missing --cov-report=html
```

Present:

```
147 passed
76% coverage
```

7.3.8 8. Sentry

Configure the environment and run:

```
python -m src.main test-sentry-command manager@epicevents.com
```

Show the issue in the Sentry dashboard.

7.4 Final repository verification

```
git status
git log --oneline --decorate --graph --all
```

Expected:

- clean working tree;
- coherent feature branches;
- no .env;
- no database file;
- no coverage artifacts;
- complete README and architecture documentation.